

Syntax Directed Definitions (SDD)

SDD is a context free grammar together with attributes and rules

Attributes are associated with grammar symbols

Rules are associated with productions

If X is a symbol and a is one of its attributes, then we write $X.a$ to denote the value of a at a particular parse tree node labelled X

Inherited and Synthesized Attributes

A Synthesized attribute

at node N is defined only in terms of attribute values at the children of N and at N itself

$E \rightarrow E + T$

An Inherited attribute

at node N is defined only in terms of attribute values at N 's parent, N itself and N 's siblings

$E' \rightarrow +TE'$

SDD of a simple desk calculator

$$L \rightarrow En$$

$$L.val = E.val$$

$$E \rightarrow E_1 + T$$

$$E.val = E_1.val + T.val$$

$$E \rightarrow T$$

$$E.val = T.val$$

$$T \rightarrow T_1 * F$$

$$T.val = T_1.val * F.val$$

$$T \rightarrow F$$

$$T.val = F.val$$

$$F \rightarrow (E)$$

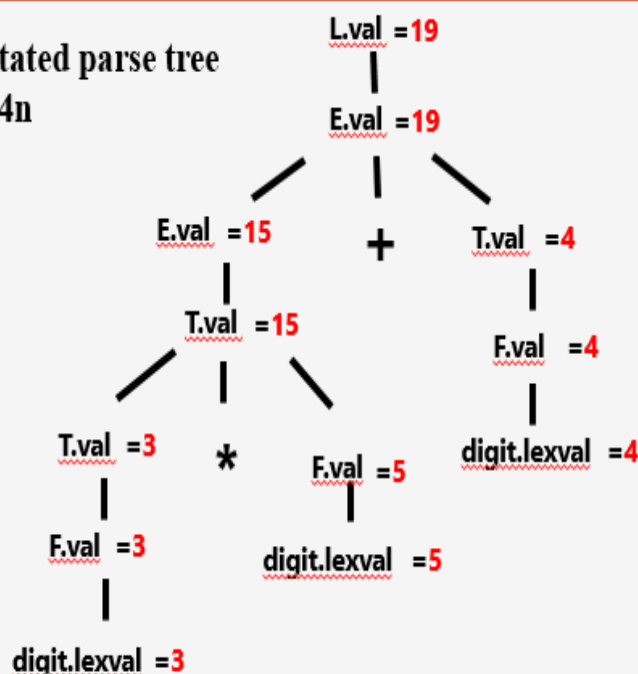
$$F.val = E.val$$

$$F \rightarrow \text{digit}$$

$$F.val = \text{digit.lexval}$$

Annotated parse tree

Annotated parse tree
3*5+4n



A parse tree, showing the value(s) of its attributes is called an annotated parse tree

$$L \rightarrow En$$

$$L.val = E.val$$

$$E \rightarrow E_1 + T$$

$$E.val = E_1.val + T.val$$

$$E \rightarrow T$$

$$E.val = T.val$$

$$T \rightarrow T_1 * F$$

$$T.val = T_1.val * F.val$$

$$T \rightarrow F$$

$$T.val = F.val$$

$$F \rightarrow (E)$$

$$F.val = E.val$$

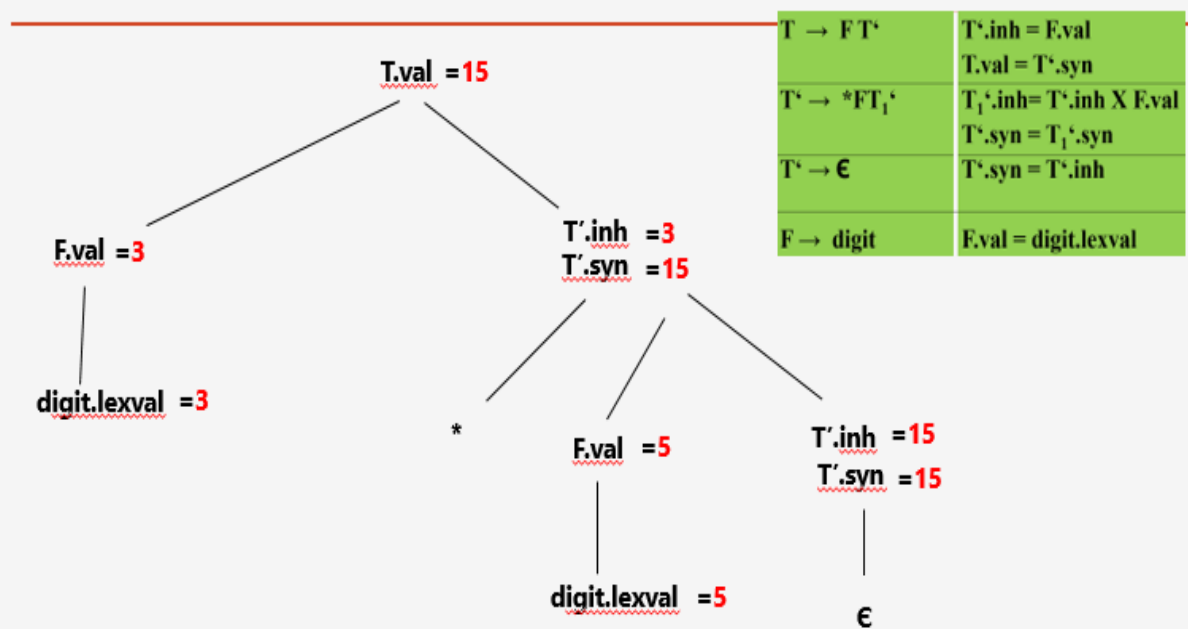
$$F \rightarrow \text{digit}$$

$$F.val = \text{digit.lexval}$$

SDD based on grammar suitable for Top down parsing

$T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

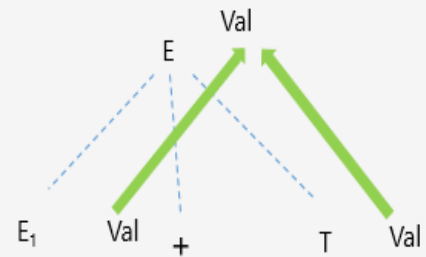
Annotated parse tree for $3 * 5$ using top down



Evaluation Orders for SDD's

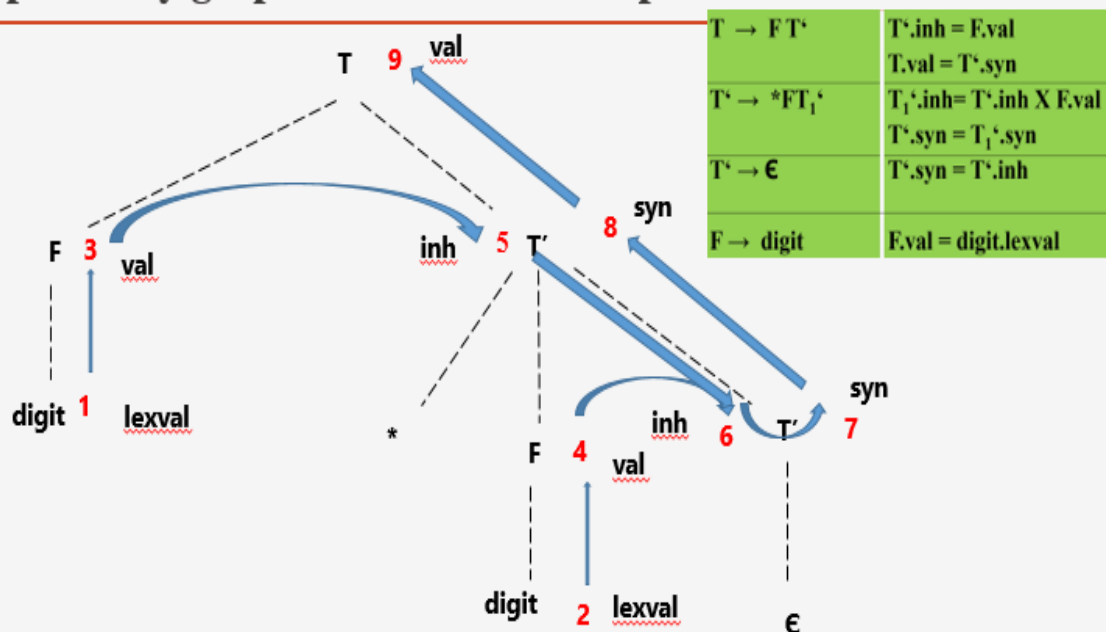
Dependency Graphs:

- It is the flow of information among the attribute instances in a particular parse tree; an edge from one attribute instance to another means that the value of the first is needed to compute the second.
- Edges express constraints implied by the semantic rules



E -> E₁ + T

Dependency graph for the annotated parse tree



S Attributed Definitions

Click to add text

An SDD is S attributed if every attribute is synthesized

L Attributed Definitions

Each attribute must be either

- 1. Synthesized or**
- 2. Inherited, but with the rules limited as follows.**

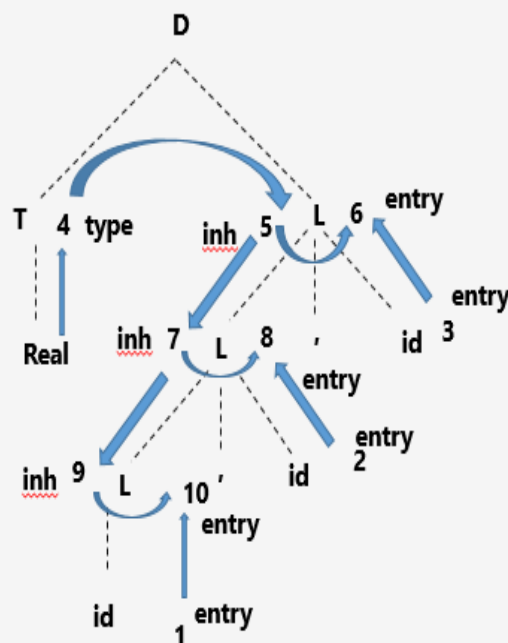
Suppose that there is a production $A \rightarrow X_1 X_2 \dots X_n$

- (a) Inherited attributes associated with the head A.**
- (b) Either Inherited or Synthesized attributes associated with the occurrences of symbols $X_1 X_2 \dots X_{i-1}$ located to the left of X_i**
- (c) Inherited or Synthesized attributes associated with this occurrence of X_i itself, but only in such a way that there are no cycles in a dependency graph formed by the attributes of this X_i**

SDD for Simple Type Declarations

$D \rightarrow T L$	$L.inh = T.val$
$T \rightarrow \text{int}$	$T.type = \text{Integer}$
$T \rightarrow \text{float}$	$T.type = \text{Float}$
$L \rightarrow L_1 , id$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$
$L \rightarrow id$	$\text{addType}(\text{id.entry}, L.inh)$

Dependency graph for a declaration `float id1,id2,id3`



$D \rightarrow T L$	$L.inh = T.val$
$T \rightarrow \text{int}$	$T.type = \text{Integer}$
$T \rightarrow \text{float}$	$T.type = \text{Float}$
$L \rightarrow L_1 , id$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$
$L \rightarrow id$	$\text{addType}(\text{id.entry}, L.inh)$